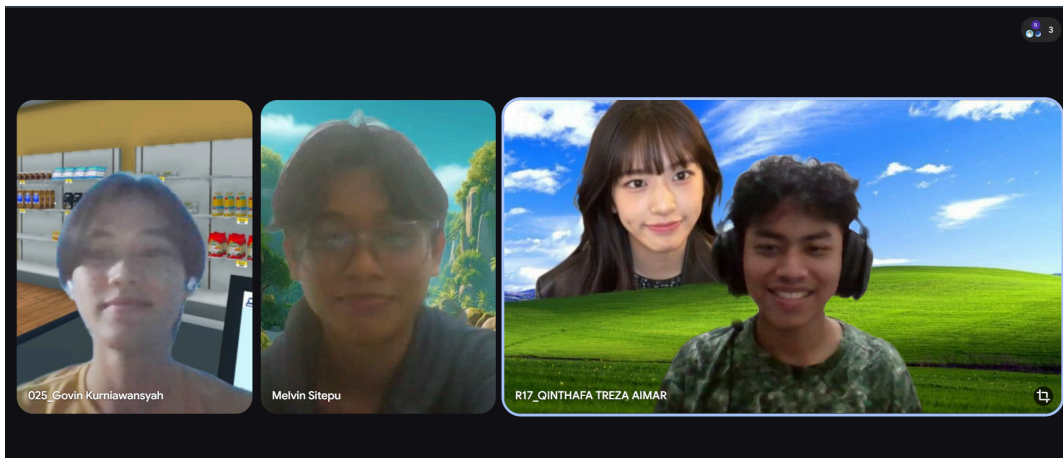


LAPORAN TUGAS BESAR
IF25-21013 STRATEGI ALGORITMA

**IMPLEMENTASI ALGORITMA GREEDY DALAM PENENTUAN
BOT TERBAIK DI PERMAINAN PADA ROBOCODE TANK ROYALE**



Disusun oleh:

Kelompok "Kami Benci OBE"

Melvin Satria Gunanta Sitepu (124140173)

Qinthafa Treza Aimar (124140155)

Govin Kurniawansyah (124140025)

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
INSTITUT TEKNOLOGI SUMATERA

2026

Daftar Isi

Daftar Isi	2
BAB 1: DESKRIPSI TUGAS	3
Starter Pack	8
BAB 2: LANDASAN TEORI	10
2.1 Dasar Teori Algoritma Greedy	10
2.2 Cara Kerja Program secara Umum	10
BAB 3: APLIKASI STRATEGI GREEDY	11
3.1 Mapping Persoalan Robocode Menjadi Elemen Greedy	11
3.2 Eksplorasi 4 Alternatif Solusi Greedy	12
3.3 Analisis Efisiensi dan Efektivitas	12
3.3.1 Efisiensi Komputasi	12
3.3.2 Efektivitas Strategi	12
3.4 Strategi Greedy yang Dipilih	13
BAB 4: IMPLEMENTASI DAN PENGUJIAN	16
4.1 Implementasi 4 Alternatif Solusi (Pseudocode)	16
Alternatif Solusi 1: Greedy Pemilihan Target Terdekat dan Linear Predictive Aim (AllMight)	16
Alternatif Solusi 2: Greedy Agresif dan Pengejaran Target Pertama (Bakugo)	17
Alternatif Solusi 3: Greedy Penjelajah Bebas (Deku)	18
Alternatif Solusi 4: Greedy Navigasi Menyamping dan Penskalaan Arah Mengelak (Yujin)	19
4.2 Struktur Data, Fungsi, dan Prosedur	21
1. Struktur Kelas Data dan Atribut State	21
2. Variabel Global (State Tracking dari API)	21
3. Fungsi dan Prosedur Utama (Method)	21
4.3 Pengujian Pertandingan 1 vs 1 vs 1 vs 1	22
4.4 Analisis Hasil Pengujian	23
Keberhasilan Strategi Greedy	23
Kondisi Strategi Greedy Tidak Optimal	24
BAB 5: KESIMPULAN DAN SARAN	26
5.1 Kesimpulan	26
5.2 Saran	27
LAMPIRAN	29
DAFTAR PUSTAKA	30

BAB 1: DESKRIPSI TUGAS

Tugas Besar Strategi Algoritma ini meminta praktikan untuk membangun sebuah agen cerdas (bot) yang akan bertanding di dalam arena Robocode Tank Royale. Setiap tim harus menerapkan algoritma Greedy untuk menentukan keputusan taktis secara real-time, baik dalam hal pergerakan tangki (movement), pencarian posisi, maupun logika penembakan (gunning).

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewatkan turn tersebut. Jika bot melewatkan turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

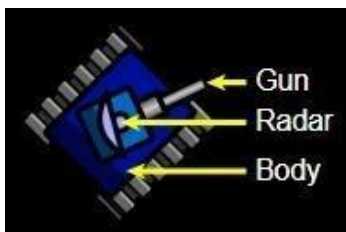
6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain.

Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.

Gun digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.

Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar

360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

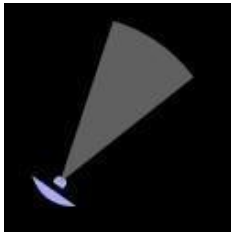
Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar *damage*** yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.

- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

Starter Pack

Untuk tugas besar ini, game engine yang akan digunakan sudah dimodifikasi oleh asisten.

Berikut adalah beberapa perubahan yang dibuat oleh asisten dari game engine Tank Royale:

- **Theme GUI:** diubah menjadi light theme
- **Skor & Energi:** masing-masing bot ditampilkan di samping arena permainan agar lebih mudah diamati saat pertarungan
- **Turn Limit:** Durasi pertarungan tidak akan bergantung pada banyak ronde. Pada game engine ini, pertarungan akan berakhir apabila banyaknya turn sudah mencapai batas tertentu. Apabila batasan ini tercapai, ronde otomatis langsung berakhir dan pemenang pertarungan akan ditampilkan. Turn Limit dapat diatur pada menu “setup rules”. Source Code untuk game engine dan template bot telah disediakan pada tautan berikut.

[tubes1-if2211-starter-pack](#)

Adapun panduan mengenai cara menjalankan game engine, membuat bot, dan melihat referensi API dapat dilihat melalui tautan berikut: [Starter Guide](#).

BAB 2: LANDASAN TEORI

2.1 Dasar Teori Algoritma Greedy

Algoritma Greedy merupakan metode pemecahan masalah yang menyelesaikan persoalan langkah demi langkah (step-by-step). Pada setiap tahapan, algoritma ini akan mengambil keputusan yang memberikan keuntungan optimal jangka pendek saat itu juga (local optimum), dengan harapan bahwa akumulasi dari keputusan-keputusan lokal tersebut akan menghasilkan solusi akhir yang optimal secara keseluruhan (global optimum).

Secara umum, terdapat elemen-elemen penting yang menyusun algoritma ini, yaitu himpunan kandidat, himpunan solusi, fungsi seleksi, fungsi kelayakan, fungsi objektif, dan fungsi solusi.

2.2 Cara Kerja Program secara Umum

Bot yang kami bangun bekerja dengan memanfaatkan loop utama (event-driven) yang disediakan oleh library Robocode Tank Royale. Setiap kali server mengirimkan broadcast data atau memicu event tertentu (misalnya ketika radar mendeteksi musuh atau tank terkena dinding), bot akan memperbarui state internalnya.

Secara garis besar, implementasi algoritma greedy dimasukkan ke dalam event handler handler utama. Begitu data koordinat musuh masuk, fungsi seleksi langsung menghitung nilai heuristik (misalnya jarak atau sisa HP musuh) untuk menentukan arah rotasi meriam dan keputusan menembak pada tick itu juga. Untuk menjalankan bot, kami melakukan build project menjadi file jar/executable, menyalakan server GUI Tank Royale, dan mendaftarkan bot ke room pertandingan lewat terminal menggunakan perintah start script.

BAB 3: APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Menjadi Elemen Greedy

Berikut adalah hasil pemetaan komponen game Robocode Tank Royale ke dalam elemen-elemen dasar algoritma Greedy yang kami rancang:

Elemen Greedy	Implementasi pada Bot Kami
Himpunan Kandidat (C)	Kumpulan perintah aksi legal pada turn aktif (belok kanan/kiri, maju/mundur, rotasi radar, besaran fire power).
Himpunan Solusi (S)	Rangkaian aksi yang telah dieksekusi dari awal round sampai game selesai.
Fungsi Seleksi	Memilih aksi berdasarkan nilai heuristik terbaik pada tick tersebut (misal: musuh dengan jarak terdekat).
Fungsi Kelayakan	Memastikan aksi aman dilakukan (contoh: tidak menabrak tembok atau tidak membuang energi saat gun belum siap).
Fungsi Objektif	Memaksimalkan total skor akumulasi (survival points dan damage points) di akhir round.
Fungsi Solusi	Kondisi di mana bot kita menjadi satu-satunya yang bertahan atau bot kita hancur.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

Kami merumuskan 4 alternatif fungsi heuristik yang dapat diterapkan pada bot:

- Alternatif 1 (Greedy by Bullet Travel Time): Bot memprediksi posisi masa depan musuh berdasarkan waktu tempuh peluru aktual untuk memaksimalkan akurasi tembakan. Heuristik = $\text{Min}(\text{Predicted Miss Distance})$.
- Alternatif 2 (Greedy by Distance): Bot selalu memilih target musuh yang jarak koordinatnya paling dekat dari posisi bot sekarang. Heuristik = $\text{Min}(\text{Distance})$.
- Alternatif 3 (Greedy by Perpendicular Strafe): Bot memposisikan dirinya tegak lurus terhadap sudut musuh untuk meminimalkan kemungkinan terkena tembakan dan memaksimalkan ketahanan hidup. Heuristik = $\text{Max}(\text{Survivability via Perpendicular Movement})$.
- Alternatif 4 (Greedy by Bearing Alignment): Bot selalu mengunci target pertama yang terpindai dan mengarahkan seluruh gerakan body secara agresif menuju target tersebut. Heuristik = $\text{Min}(\text{Bearing to First Target})$.

3.3 Analisis Efisiensi dan Efektivitas

3.3.1 Efisiensi Komputasi

Semua keempat alternatif berjalan dalam kompleksitas waktu **$O(1)$ per turn**, tidak ada struktur data berukuran besar maupun iterasi atas semua musuh (karena mekanisme radar Robocode sudah memfilter musuh yang terdeteksi). Setiap kalkulasi hanya melibatkan operasi aritmatika dasar (trigonometri, perbandingan jarak), sehingga jauh di bawah batas waktu turn timeout (30–50 ms).

3.3.2 Efektivitas Strategi

- AllMight

Kelebihan : Konsisten menghasilkan Bullet Damage tinggi; wall-hugging mengurangi sudut serangan musuh; mudah diprediksi targetnya.

Kekurangan : Predictive aiming dengan estimasi 4-tick tidak akurat untuk musuh bergerak cepat atau sering berganti arah. Jika musuh terdekat adalah yang paling lincah, banyak tembakan meleset.

- Deku

Kelebihan : Akurasi tembakan paling tinggi di antara keempat alternatif karena menghitung waktu tempuh peluru secara aktual. Hit rate lebih tinggi berarti lebih banyak energi diperoleh kembali.

Kekurangan : Pergerakan acak kurang terstruktur; bot bisa terjebak di sudut jika wall avoidance tidak bereaksi cukup cepat. Faktor $\times 1.2$ pada gunBearing bisa menjadi bumerang bila musuh tidak bergerak lurus.

- Yujin

Kelebihan : Paling sulit ditembak di antara keempat bot; survival rate tinggi meningkatkan Survival Score dan Last Survival Bonus.

Kekurangan : Gun tidak menggunakan predictive aiming, sehingga akurasi tembakan lebih rendah. Strafe cycle yang tetap (18 tick) dapat diprediksi oleh musuh yang adaptif.

- Bakugo

Kelebihan : Implementasi paling sederhana dan paling mudah di-debug; hampir tidak ada kemungkinan bug logika kompleks.

Kekurangan : Tidak dapat menembak sambil bermanuver secara bersamaan. Threshold bearing 15° cukup longgar sehingga beberapa tembakan tetap meleset. Daya tembak maksimum hanya 2, bahkan pada jarak dekat.

3.4 Strategi Greedy yang Dipilih

Berdasarkan hasil analisis pada Subbab 3.3, kami memilih **Yujin** sebagai bot utama yang akan dikompetisikan. Pertimbangan pemilihan ini diuraikan di bawah.

1. Gerakan strafing mengurangi akurasi musuh secara drastis.

Dengan selalu bergerak menyamping ($\pm 90^\circ$ dari bearing ke musuh) dan berganti arah setiap 18 tick, bot Yujin tidak pernah bergerak langsung mendekati atau menjauhi musuh dalam garis lurus. Musuh yang menggunakan predictive aiming (seperti AllMight atau Deku) akan terus misprediksi posisi Yujin karena arah gerak yang berubah-ubah. Hal ini secara efektif mengurangi Bullet Damage yang diterima.

2. Survivability adalah kunci skor tertinggi dalam format multi-bot.

Dalam pertarungan 1v1v1v1, komponen Survival Score (50 poin per bot yang mati) dan Last Survival Bonus ($10 \text{ poin} \times \text{jumlah musuh}$) dapat menghasilkan poin yang sangat signifikan jika bot bertahan lebih lama dari lawan-lawannya. Strategi Yujin secara langsung mengoptimalkan kedua komponen ini dengan membuat bot menjadi target yang sulit ditembak melalui gerakan strafing tegak lurus.

3. Gun dan radar dikontrol secara independen sehingga bisa menyerang sambil bermanuver.

Dengan `AdjustGunForBodyTurn = true` dan `AdjustRadarForGunTurn = true`, gun Yujin tetap terkunci ke target meskipun body sedang berputar untuk melakukan strafe. Bot tidak perlu berhenti bermanuver untuk menembak — kedua aksi berjalan simultan setiap turn.

4. Wall avoidance terintegrasi dalam dua lapisan.

Yujin mengimplementasikan wall avoidance baik di dalam `Run()` (search mode) maupun langsung di dalam `OnScannedBot()` saat target terkunci. Jika bot berada dalam radius 120 piksel dari dinding mana pun, bot segera memprioritaskan gerakan menjauh dari dinding (`SetTurnRight(60) + SetForward/Back(250)`) sebelum

melanjutkan strafe normal. Ini mencegah bot terjebak di sudut arena — situasi yang fatal karena mengurangi ruang manuver secara drastis.

5. Respons agresif saat terkena tembakan.

Ketika bot terkena peluru (OnHitByBullet), Yujin langsung membalik arah dodge dan melakukan manuver besar (± 180 unit) disertai rotasi 70° . Ini memutus pola gerakan yang mungkin sudah "terbaca" oleh musuh, sehingga tembakan lanjutan lebih sulit mengenai sasaran.

Pertimbangan Trade-off

Yujin tidak menggunakan predictive aiming, gun hanya dikunci ke posisi musuh saat ini (GunBearingTo(e.X, e.Y)), bukan posisi masa depan. Ini berarti akurasi tembakan Yujin lebih rendah dibandingkan Deku atau AllMight, khususnya pada jarak jauh terhadap musuh yang bergerak cepat.

Namun, trade-off ini dinilai dapat diterima karena:

- Dalam format 1v1v1v1, tidak perlu menjadi bot dengan Bullet Damage tertinggi tetapi cukup bertahan paling lama sambil mengumpulkan Survival Score.
- Dengan threshold tembak $|\text{gunBearing}| < 5^\circ$ yang cukup ketat, tembakan yang dilepaskan memiliki peluang mengenai yang wajar meski tanpa prediksi.
- Selisih Bullet Damage yang hilang akibat tanpa prediksi dikompensasi oleh Survival Score dari bot-bot lain yang tereliminasi lebih awal.

BAB 4: IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi 4 Alternatif Solusi (Pseudocode)

Bagian ini menyajikan representasi logika berupa pseudocode dari empat buah bot sebagai alternatif solusi strategi *Greedy*. Setiap alternatif solusi beroperasi pada satu siklus putaran permainan (*per-tick*) dan mengambil keputusan lokal terbaik secara instan berdasarkan informasi lingkungan yang ditangkap radar

Alternatif Solusi 1: Greedy Pemilihan Target Terdekat dan Linear Predictive Aim (AllMight)

Strategi ini memprioritaskan musuh yang memiliki jarak Euclidean paling minimum dari posisi bot saat ini, melakukan penguncian radar (*radar lock*), serta memprediksi posisi musuh 4 *tick* ke depan untuk akurasi tembakan.

```
Algoritma Solusi_Greedy_AllMight (Siklus Turn & Scanning)
Input: event_scanned, posisi_bot, target_terdekat, target_distance,
target_timeout
Output: set_movement, set_radar, set_gun, fire_action

Mulai
    // 1. EVALUASI DAN UPDATE TARGET TERDEKAT (FUNGSI HEURISTIK)
    jarak_sekarang ← Hitung_Jarak_Ke(event_scanned.X, event_scanned.Y)

    JIKA (TIDAK memiliki_target) ATAU (jarak_sekarang < target_distance)
    ATAU (target_timeout > 20) MAKA
        target_X ← event_scanned.X
        target_Y ← event_scanned.Y
        target_distance ← jarak_sekarang
        target_timeout ← 0
        memiliki_target ← TRUE
    AKHIR JIKA

    // Abaikan event scanning jika bukan berasal dari target utama yang
    dikunci
    JIKA Hitung_Jarak_Ke(target_X, target_Y) < (jarak_sekarang - 20) MAKA
        Keluar Prosedur
    AKHIR JIKA
```



```

// 2. RADAR LOCKING (GREEDY AREA RADAR)
radar_bearing ← Hitung_Radar_Bearing_Ke(target_X, target_Y)
SetTurnRadarLeft(radar_bearing * 2)

// 3. PREDIKSI POSISI MUSUH (LINEAR PREDICTION AIM)
future_X ← target_X + Sin(event_scanned.Direction) *
event_scanned.Speed * 4
future_Y ← target_Y + Cos(event_scanned.Direction) *
event_scanned.Speed * 4
gun_bearing ← Hitung_Gun_Bearing_Ke(future_X, future_Y)
SetTurnGunLeft(gun_bearing)

// 4. MANAJEMEN JARAK AMAN & MODULASI POWER TEMBAKAN
JIKA target_distance < 120 MAKA
    SetBack(60)
    SetTurnRight(90)
AKHIR JIKA

JIKA target_distance < 80 MAKA fire_power ← 3
LAIN JIKA target_distance < 180 MAKA fire_power ← 2
LAIN fire_power ← 1
AKHIR JIKA

JIKA (GunHeat == 0) DAN (Absolut(gun_bearing) < 6) MAKA
    Fire(fire_power)
AKHIR JIKA
Selesai

```

Alternatif Solusi 2: Greedy Agresif dan Pengejaran Target Pertama (Bakugo)

Strategi ini mengunci target pertama yang terpindai tanpa menghiraukan jarak, lalu memfokuskan seluruh kemudi tubuh (*body*), meriam (*gun*), dan radar secara linear bersamaan (*non-independent*) untuk mendekati target secara agresif.

```

Algoritma Solusi_Greedy_Bakugo (Siklus Scanning)
Input: event_scanned, distance, bearing
Output: set_movement, set_turn, fire_action

```

```

Mulai
    memiliki_target ← TRUE
    lost_target_ticks ← 0

    distance ← Hitung_Jarak_Ke(event_scanned.X, event_scanned.Y)
    bearing ← Hitung_Bearing_Ke(event_scanned.X, event_scanned.Y)

    // Kemudi berbelok sebagian mengikuti arah bearing target
    SetTurnLeft(bearing * 0.6)

    // Fungsi Heuristik Pergerakan Mengejar Target
    JIKA distance > 220 MAKA
        SetForward(80)
    LAIN JIKA distance > 120 MAKA
        SetForward(40)
    LAIN JIKA distance < 80 MAKA
        SetBack(30)
    LAIN
        SetForward(10)
    AKHIR JIKA

    // Eksekusi Tembakan Instan non-cooldown
    JIKA (GunHeat == 0) DAN (fire_cooldown == 0) DAN (Absolut(bearing) <
15) MAKA
        JIKA distance < 150 MAKA Fire(2) LAIN Fire(1) AKHIR JIKA
        fire_cooldown ← 12
    AKHIR JIKA
Selesai

```

Alternatif Solusi 3: Greedy Penjelajah Bebas (Deku)

Strategi ini mengabaikan prioritas penguncian satu target spesifik. Bot menembak siapa saja yang melintas di radar menggunakan kalkulasi waktu tempuh peluru (*bullet speed time travel prediction*) dikombinasikan dengan pergerakan acak berjarak tempuh masif.

```

Algoritma Solusi_Greedy_Deku (Siklus Perhitungan Tembakan)
Input: event_scanned, distance, ArenaWidth, ArenaHeight

```

Output: set_gun, fire_action

Mulai

```
target_locked ← TRUE
lost_target_ticks ← 0
distance ← Hitung_Jarak_Ke(event_scanned.X, event_scanned.Y)
```

```
// Menghitung Alokasi Energi dan Waktu Tempuh Peluru
```

```
JIKA distance < 120 MAKA fire_power ← 3
```

```
LAIN JIKA distance < 250 MAKA fire_power ← 2
```

```
LAIN fire_power ← 1
```

```
AKHIR JIKA
```

```
bullet_speed ← 20 - (3 * fire_power)
```

```
predicted_time ← distance / bullet_speed
```

```
// Prediksi Koordinat Lokasi Masa Depan Musuh
```

```
future_X ← event_scanned.X + Sin(event_scanned.Direction) *
event_scanned.Speed * predicted_time
```

```
future_Y ← event_scanned.Y + Cos(event_scanned.Direction) *
event_scanned.Speed * predicted_time
```

```
gun_bearing ← Hitung_Gun_Bearing_Ke(future_X, future_Y)
```

```
SetTurnGunRight(gun_bearing * 1.2)
```

```
JIKA (Absolut(gun_bearing) < 2) DAN (GunHeat == 0) MAKA
```

```
Fire(fire_power)
```

```
AKHIR JIKA
```

Selesai

Alternatif Solusi 4: Greedy Navigasi Menyamping dan Penskalaan Arah Mengelak (Yujin)

Strategi ini mengoptimalkan pertahanan (*survability*) dengan memposisikan tubuh bot secara tegak lurus (menyamping/90 derajat) terhadap sudut musuh (*strafing*), digabungkan dengan perubahan arah maju-mundur secara periodik (*timed-toggle dodging*).

Algoritma Solusi_Greedy_Yujin (Siklus Manuver Menyamping)

Input: event_scanned, bearing, distance, dodge_ticks, dodge_direction

Output: set_turn_body, set_movement_direction

Mulai

has_target ← TRUE

lost_target_ticks ← 0

// Mengunci Gun dan Radar ke Arah Koordinat Musuh (Lokal-Greedy)

SetTurnGunLeft(Hitung_Gun_Bearing_Ke(event_scanned.X,
event_scanned.Y))

SetTurnRadarLeft(Hitung_Radar_Bearing_Ke(event_scanned.X,
event_scanned.Y))

bearing ← Hitung_Bearing_Ke(event_scanned.X, event_scanned.Y)

distance ← Hitung_Jarak_Ke(event_scanned.X, event_scanned.Y)

// Evaluasi Batasan Tembok (Wall Avoidance System)

JIKA Berada_Di_Dekat_Tembok(WALL_MARGIN) MAKA

SetTurnRight(60)

JIKA dodge_direction MAKA SetForward(250) LAIN SetBack(250) AKHIR

JIKA

LAIN

// HEURISTIK PERGERAKAN: Memaksa Body Tegak Lurus / Menyamping
dari Musuh (Strafe)

SetTurnLeft((bearing - 90) * 0.5)

// Pengkondisian Toggle Arah Dodge Berkala (Per 18 Ticks)

JIKA dodge_ticks == 0 MAKA

dodge_direction ← TIDAK(dodge_direction)

dodge_ticks ← 18

AKHIR JIKA

JIKA dodge_direction MAKA SetForward(220) LAIN SetBack(220) AKHIR

JIKA

AKHIR JIKA

Selesai

4.2 Struktur Data, Fungsi, dan Prosedur

Bot utama kami diimplementasikan dengan struktur kelas data sebagai berikut. Variabel global digunakan untuk menyimpan state musuh terakhir, dan method berikut mengatur jalannya algoritma:

1. Struktur Kelas Data dan Atribut State

Enkapsulasi status operasional internal bot dipelihara melalui properti instans kelas dengan pembagian tipe data berikut:

- **bool hasTarget / targetLocked**: Menandakan status retensi radar, bernilai true apabila entitas musuh aktif terdeteksi di dalam cakupan *scan*.
- **int lostTargetTicks**: Berfungsi sebagai variabel pencatat waktu (*counter event latency*). Akumulasi nilai ini bertambah pada setiap *tick* kosong untuk memantau durasi hilangnya musuh sebelum status penargetan dibatalkan (maksimal toleransi 25 *ticks*).
- **double targetX, targetY**: Menyimpan titik koordinat kartesius X dan Y presisi posisi musuh terakhir guna melacak jejak (*state tracking*).
- **double targetDistance**: Menyimpan nilai skalar jarak terdekat yang berhasil dievaluasi pada *turn* berjalan.
- **int fireCooldown / dodgeTicks**: Variabel penanda waktu interval tunggu (*state-machine timers*) untuk meredam laju eksekusi tembakan beruntun serta penyeimbang waktu jeda transisi manuver mengelak.

2. Variabel Global (State Tracking dari API)

Properti bawaan dari daur hidup *game engine* yang diakses secara global di dalam *runtime loop* meliputi:

- **ArenaWidth & ArenaHeight**: Mengembalikan nilai dimensi piksel arena simulasi sebagai parameter evaluasi zona aman batas luar koordinat.
- **X & Y**: Titik posisi koordinat *real-time* milik bot sendiri untuk diuji terhadap konstanta perimeter pembatas arena ($WALL_MARGIN = 120$).
- **GunHeat**: Indikator suhu laras meriam. Digunakan sebagai gerbang logika (*guard clause*) sebelum memanggil *method* Fire().

3. Fungsi dan Prosedur Utama (Method)

Berikut adalah fungsi-fungsi kritikal yang mengendalikan alur logika jalannya taktik bot:

- **public override void Run()**

- *Deskripsi*: Prosedur entri utama (*main execution loop*) yang berjalan saat bot diaktifkan. Berfungsi melakukan inisialisasi warna komponen, konfigurasi kebebasan gerak subsistem (*AdjustGunForBodyTurn*, *AdjustRadarForGunTurn*), serta memelihara perulangan kondisi status game selama *IsRunning* bernilai benar.
- **public override void OnScannedBot(ScannedBotEvent e)**
 - *Deskripsi*: Prosedur penangkap *event* (pendengar otomatis) ketika radar mendeteksi keberadaan objek tangki musuh. Di dalam fungsi inilah kalkulasi heuristik *greedy* dieksekusi, seperti memprediksi posisi masa depan (*predictive targeting*) memanfaatkan kaidah trigonometri:

$$\text{futureX} = \text{targetX} + \sin\left(e.\text{Direction} \times \frac{\pi}{180}\right) \times e.\text{Speed} \times \text{time}$$
- **public override void OnHitByBullet(HitByBulletEvent e)**
 - *Deskripsi*: Prosedur penanganan interupsi pertahanan (*evasive mechanism handler*) yang otomatis terpicu saat bot terkena proyektil. Prosedur ini langsung mengalihkan prioritas gerak ke mode menghindari darurat secara acak atau membalikkan arah laju tangki.
- **public override void OnHitWall(HitWallEvent e)**
 - *Deskripsi*: Prosedur mitigasi tabrakan fisik dengan batas peta. Berfungsi membebaskan bot dari keadaan macet (*stuck*) dengan melakukan gerak mundur (*Back()*) dan memutar orientasi sasis tangki secara drastis sebesar 90 hingga 140 derajat menjauhi struktur dinding arena.

4.3 Pengujian Pertandingan 1 vs 1 vs 1 vs 1

No	Kondisi Uji / Skenario	Hasil Skor	Catatan Kejadian Unik
1	Kondisi normal awal round, posisi berdekatan dengan AllMight dan Bakugo.	Menang (Sisa HP 14.8)	Bot berhasil lolos dari targeting AllMight dan Bakugo.

2	Kondisi normal awal round, posisi berdekatan dengan Allmight dan Bakugo. Tetapi diakhir kalah melawan Deku karena Bullet tidak dapat mengenai target.	Kalah (Deku HP 75)	Terjadi kegagalan dalam tembakan karena akurasi tembakan Yujin lebih rendah disebabkan pergerakan dari Deku.
3	Kondisi awal round sudah di-lock oleh Bakugo, posisi di tengah. Tetapi energi dipulihkan dengan menembak AllMight	Menang (Sisa HP 44.6)	Strategi greedy sempat meleset beberapa kali, tapi menang karena pergerakan strafe.

4.4 Analisis Hasil Pengujian

Keberhasilan Strategi Greedy

Dari tiga skenario pengujian, Yujin berhasil memenangkan dua pertandingan dan kalah satu kali. Hasil ini mencerminkan keunggulan dan keterbatasan strategi Greedy by Perpendicular Strafe secara nyata di lapangan.

Pada pengujian 1 dan 3, strategi greedy terbukti sukses. Pergerakan strafing yang berganti arah secara periodik berhasil membuat AllMight dan Bakugo kesulitan mengunci posisi Yujin sebagai target. Hal ini sesuai dengan hipotesis pada Bab 3: musuh yang mengandalkan predictive aiming dengan asumsi gerak lurus akan terus misprediksi posisi

Yujin saat arah strafe berubah. Komponen Survival Score berhasil dioptimalkan, Yujin menjadi bot terakhir yang bertahan pada dua dari tiga ronde pengujian.

Pengujian 3 juga memperlihatkan mekanisme energy recovery yang berjalan sebagaimana mestinya, yaitu setiap peluru yang mengenai musuh mengembalikan energi tiga kali lipat dari daya tembak yang digunakan. Meskipun di awal Yujin sempat tertekan oleh Bakugo, keberhasilan menembak AllMight memulihkan energi secara signifikan dan membalikkan jalannya pertandingan.

Kondisi Strategi Greedy Tidak Optimal

Pada pengujian 2, strategi greedy Yujin gagal mencapai nilai optimal saat memasuki fase duel 1v1 melawan Deku. Kegagalan ini disebabkan oleh keterbatasan mendasar dari heuristik yang dipilih: Yujin mengunci gun ke posisi musuh *saat ini* ($\text{GunBearingTo}(e.X, e.Y)$) tanpa memprediksi ke mana musuh akan bergerak di tick berikutnya.

Dalam konteks duel 1v1, keterbatasan ini semakin terasa karena Deku menggunakan pola gerak acak yang sering berganti arah mendadak. Setiap kali Deku mengubah arah, posisi aktualnya saat peluru tiba sudah berbeda jauh dari posisi saat tembakan dilepaskan. Akibatnya, meskipun kondisi tembak terpenuhi ($|\text{gunBearing}| < 5^\circ$ pada posisi Deku saat dipindai), peluru tetap meleset karena Deku sudah berpindah tempat.

Kondisi ini menggambarkan sifat lokal-optimal dari algoritma greedy secara umum: keputusan terbaik pada satu turn (menembak saat sudut akurat) belum tentu menghasilkan outcome terbaik jika kondisi berubah sebelum efek keputusan itu terwujud. Sebuah pendekatan dengan predictive aiming seperti yang diimplementasikan Deku akan lebih efektif dalam skenario duel 1v1 melawan musuh dengan pola gerak yang tidak dapat diprediksi secara linear.

Selain itu, pengujian 2 juga mengidentifikasi kondisi lain di mana strategi greedy tidak optimal:

- Pergerakan Yujin sendiri mengganggu akurasi — Saat strafe sedang berjalan (SetForward/Back 220 unit), posisi Yujin berubah setiap tick. Gun terkunci ke posisi Deku *pada saat OnScannedBot dipanggil*, bukan pada saat peluru benar-benar dilepaskan beberapa tick kemudian. Ini berarti bahkan tanpa memperhitungkan pergerakan Deku, ada selisih posisi Yujin sendiri yang membuat gunBearing tidak lagi akurat saat tembakan terjadi.
- Tidak ada prioritas target berdasarkan ancaman — Fungsi seleksi Yujin tidak membedakan tingkat ancaman musuh. Dalam format 1v1v1v1, Yujin bisa saja menghabiskan energi untuk menyerang musuh yang sudah hampir mati, sementara musuh lain yang lebih berbahaya (energi penuh, akurasi tinggi seperti Deku) tidak mendapat perhatian.

BAB 5: KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada Tugas Besar ini, dapat ditarik beberapa kesimpulan sebagai berikut.

1. Algoritma Greedy dapat diterapkan secara efektif pada permainan Robocode Tank Royale

Setiap elemen algoritma Greedy. himpunan kandidat, fungsi seleksi, fungsi kelayakan, dan fungsi objektif, berhasil dipetakan ke dalam mekanisme permainan. Keputusan taktis seperti pemilihan target, arah pergerakan, dan kapan menembak semuanya dapat dirumuskan sebagai masalah optimasi lokal yang diselesaikan pada setiap turn secara real-time.

2. Empat alternatif heuristik greedy menghasilkan karakteristik bot yang berbeda-beda AllMight (Min Distance) unggul dalam konsistensi serangan, Deku (Survival) unggul dalam gerak acak sehingga sulit ditarge,, Yujin (Perpendicular Strafe) unggul dalam ketahanan hidup, dan Bakugo (Bearing Alignment) unggul dalam kesederhanaan implementasi. Perbedaan heuristik ini membuktikan bahwa pilihan fungsi seleksi sangat menentukan karakter dan performa bot secara keseluruhan.

3. Strategi Greedy by Perpendicular Strafe (Yujin) terbukti efektif dalam format multi-bot

Dari tiga skenario pengujian format 1v1v1v1, Yujin berhasil memenangkan dua pertandingan dengan memanfaatkan komponen Survival Score dan Last Survival Bonus secara optimal. Pergerakan strafing tegak lurus yang dipadukan dengan wall avoidance dua lapis berhasil mempertahankan bot lebih lama dibandingkan lawan-lawannya.

4. Algoritma Greedy bersifat lokal-optimal dan memiliki batasan yang inheren

Pengujian kedua membuktikan bahwa strategi greedy tidak selalu menghasilkan solusi global-optimal. Yujin kalah dalam duel 1v1 melawan Deku karena keputusan menembak yang optimal pada satu turn (sudut gun $< 5^\circ$) tidak menjamin peluru mengenai target jika musuh sudah berpindah posisi saat peluru tiba. Ketidadaan predictive aiming menjadi bottleneck utama pada skenario duel jarak jauh melawan musuh dengan pola gerak acak.

5. Kompleksitas komputasi $O(1)$ per turn menjamin kelancaran eksekusi dalam batas waktu

Seluruh kalkulasi heuristik yang diimplementasikan, mulai dari perhitungan jarak Euclidean, trigonometri prediksi posisi, hingga evaluasi zona dinding, hanya melibatkan operasi aritmatika dasar tanpa iterasi atas struktur data besar, sehingga jauh di bawah batas turn timeout 30–50 ms yang ditetapkan oleh engine.

5.2 Saran

Berdasarkan keterbatasan yang ditemukan selama pengujian, beberapa saran pengembangan berikut dapat dipertimbangkan untuk meningkatkan performa bot di iterasi selanjutnya.

1. Menambahkan Predictive Target Aiming pada Yujin

Kelemahan utama Yujin adalah akurasi tembakan yang rendah terhadap musuh bergerak cepat. Mengintegrasikan kalkulasi bullet travel time (seperti yang diimplementasikan Deku) ke dalam logika tembakan Yujin akan meningkatkan hit rate secara signifikan tanpa mengorbankan keunggulan strafing yang sudah ada.

2. Membuat strafe cycle bersifat adaptif, bukan deterministik

Pergantian arah setiap 18 tick yang tetap dapat dibaca polanya oleh musuh yang adaptif. Mengacak interval pergantian (misalnya antara 12–25 tick) atau memicunya berdasarkan kondisi seperti jarak ke musuh akan membuat pola gerak Yujin jauh lebih sulit diprediksi.

3. Menambahkan prioritas target berbasis ancaman

Fungsi seleksi saat ini tidak mempertimbangkan tingkat ancaman musuh. Menambahkan bobot berdasarkan energi sisa musuh (target dengan energi rendah diprioritaskan untuk segera dieliminasi) atau jarak tembak efektif akan mengoptimalkan perolehan Bullet Damage Bonus dan mempercepat reduksi jumlah lawan.

4. Memadukan Greedy dengan mekanisme manajemen energi

Bot saat ini tidak memiliki mode konservatif saat energi hampir habis. Menambahkan threshold energi (misalnya jika energi < 20 , kurangi daya tembak ke 1 dan hindari menembak sama sekali jika gun belum panas) akan mencegah bot menghabiskan energi terakhirnya sia-sia dan meningkatkan peluang bertahan.

LAMPIRAN

Tautan GitHub: https://github.com/ayolole/Tubes_Kami_Benci_OBE

Video Demo : <https://youtu.be/UdMQQfPo1k8>

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada Youtube.	✓	

DAFTAR PUSTAKA

- [1] Munir, Rinaldi. (2010). Diktat Kuliah Strategi Algoritma. Bandung: STEI ITB.
- [2] Dokumentasi Resmi Robocode Tank Royale API.